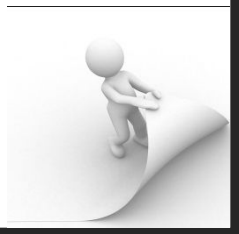
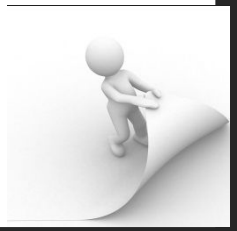
C++11新特性之二：提升类型安全及内存管理

张锦成 2015/10/13



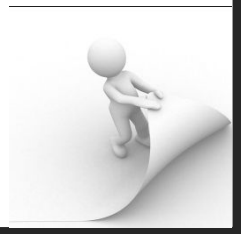
Outline

- 强类型枚举
- 空指针nullptr
- 堆内存管理：智能指针
- 堆内存管理：垃圾回收机制



1

强类型枚举



强类型枚举

回顾C++98枚举的使用

```
enum Gender{
    Male,
    Female
};

enum {
    Boy,
    Girl
};

Gender g = Male;
Gender e = Female;
std::cout << "g=" << g << " e=" << e << std::endl;
std::cout << Boy << " " << Girl << std::endl;
```

强类型枚举

- C++98枚举的问题：enum是非强作用域类型

```
enum Type{General, Light};

enum Category{General, Cannon};
```

强类型枚举

C++98枚举的问题：隐式转换为整型

```
enum Type{General, Light};
enum Category{Pistol, Cannon};

struct Killer{
    Killer(Type t, Category c): type(t), category(c){}

    Type type;
    Category category;
};
Killer cool(General, Pistol);

//... many lines of code

if(cool.type >= Pistol) {
    std::cout << ">=" << std::endl;
}
```

强类型枚举

C++98枚举的问题：占用存储空间和符号性的不确定

```
enum C {C1=1, C2=2};
enum D {D1=1, D2=2, Dbig = 0xFFFFFFF0};
enum E {E1=1, E2=2, Ebig = 0xFFFFFFFF};

int main()
{
    std::cout << sizeof(C1) << std::endl; // 4

    std::cout << Dbig << std::endl; // 4294967280
    std::cout << sizeof(D1) << std::endl; // 4
    std::cout << sizeof(Dbig) << std::endl; // 4

    std::cout << Ebig << std::endl; // 68719476735
    std::cout << sizeof(Ebig) << std::endl; // 8

    return 0;
}
```

强类型枚举

C++11使用了强类型枚举，同时兼容C++98枚举

强作用域：不再是全局的

转换限制：不能隐式转换为整型

可以指定底层类型：默认为int，可以显示指定为除wchar_t以外的任何整型

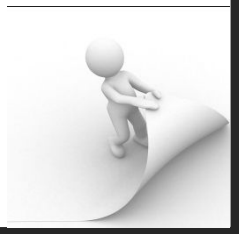
强类型枚举

C++11使用了强类型枚举，同时兼容C++98枚举

```
enum class Type{General, Light, Medium, Heavy};
enum class Category: int{General = 1, Pistol, MachineGun, Cannon};
Type t = Type::Light;
t = General; // error here, 必须指定强类型名称

if(t == Type::General){
    std::cout << "General Weapon" << std::endl;
}
if(t == Category::General){  // error here, 必须是同类型比较
    std::cout << "General Weapon" << std::endl;
}
if(t>0) { // error here, 不允许隐式类型转换
    std::cout << "General Weapon" << std::endl;
}
if((int)t>0) {
    std::cout << "General Weapon" << std::endl;
}
```

2

空指针nullptr



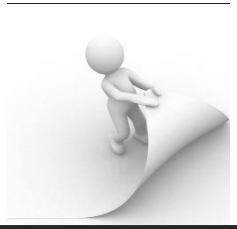
空指针nullptr

C++98中NULL的二义性，下面是一个VS的例子

```
/*·Define·NULL·pointer·value·*/
#ifndef·NULL
#ifdef·__cplusplus
#define·NULL····0
#else
#define·NULL····((void·*)0)
#endif
#endif
```

```
void f(char* c) {
    std::cout << "char*" << std::endl;
}

void f(int i) {
    std::cout << "int" << std::endl;
}

f(NULL);
```

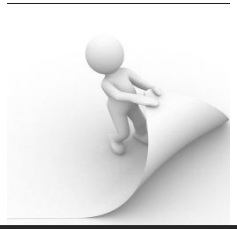
Visual Studio, G++编译器处理方式不一样



空指针nullptr

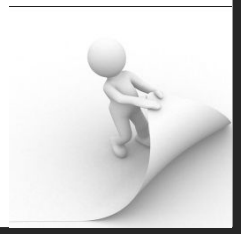
- C++11使用nullptr表示空指针，同时兼容NULL，但不建议使用

```
void f(char* c) {
    std::cout << "char*" << std::endl;
}

void f(int i) {
    std::cout << "int" << std::endl;
}

f(nullptr);
```



3

堆内存管理：智能指针

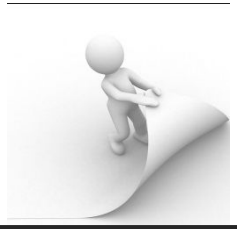


智能指针

- auto_ptr: C++98中的智能指针， C++11中被放弃

```
void f(std::auto_ptr<int>ap)
{
    std::cout<<*ap;
}

std::auto_ptr<int>ap1(new int(0));
f(ap1);
std::cout << *ap1 << std::endl; // error here
```

```
int *a = new int[10];

std::auto_ptr<int> ap(a);
```



智能指针

unique_ptr指针唯一地拥有所指向的内存

不能与其他unique_ptr指针共享内存

仅能通过move函数转移对象所有权，转移所有权之后其值变为nullptr

```
std::unique_ptr<int> up1{new int(12)};
std::cout << *up1 << std::endl;

std::unique_ptr<int> up2 = move(up1);
std::cout << *up2 << std::endl;
std::cout << (up1==nullptr) << std::endl;
std::cout << (up2==nullptr) << std::endl;
```

- shared_ptr指针利用引用计数进行内存管理

```
std::shared_ptr<int> sp1{new int{22}};
std::shared_ptr<int> sp2{sp1};
std::cout << *sp1 << std::endl;
std::cout << *sp2 << std::endl;

sp1.reset();
std::cout << (sp1 == nullptr) << std::endl;
std::cout << *sp2 << std::endl;
```

weak_ptr指针不参与引用计数

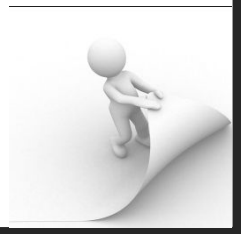
weak_ptr指向shared_ptr指针指向的内存，但是不拥有该内存

weak_ptr可以根据lock()返回值来检验shared_ptr指针的有效性

```
std::shared_ptr<int> sp1{new int{22}};
std::weak_ptr<int> wp{sp1};
std::shared_ptr<int> wps = wp.lock();
std::cout << *wps << std::endl;

std::shared_ptr<int>sp(new int(0));
std::weak_ptr<int>ap(sp);
sp.reset();
std::cout << (ap.lock() == nullptr)<< std::endl;
```

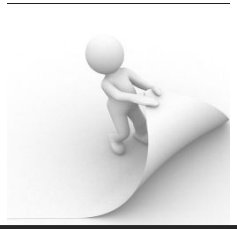
4

堆内存管理：垃圾回收机制



垃圾回收机制

- 标记-清除(Mark-Sweep)
- 标记-整理(Mark-Compact)
- 标记-拷贝(Mark-Copy)



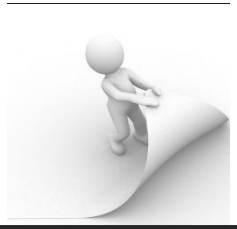
垃圾回收机制

标记-清除(Mark-Sweep)

标记后

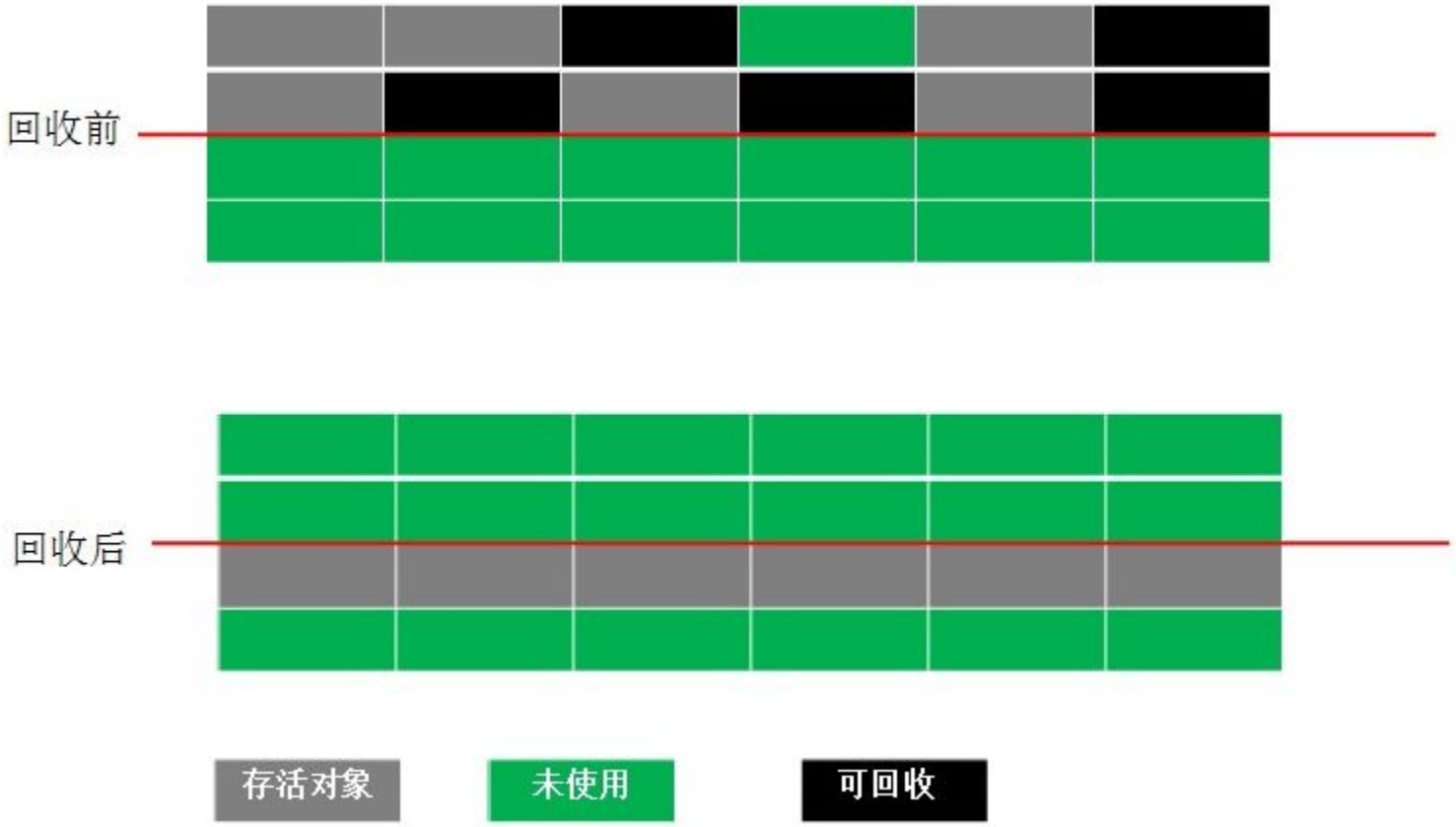
清除后

存活对象　未使用　可回收



垃圾回收机制

标记-拷贝(Mark-Copy)

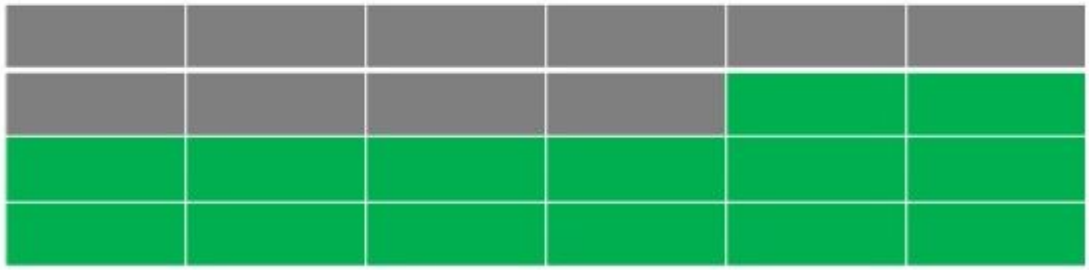


垃圾回收机制

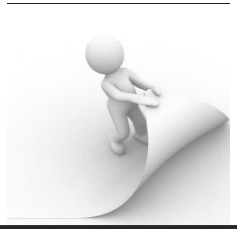
标记-整理(Mark-Compact)

回收前

回收后



存活对象　未使用　可回收

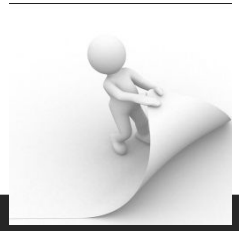


References

1. 《深入理解C++11: C++11新特性解析与应用》第5章

2. C++11新特性: http://www.cocos.com/doc/tutorial/show?id=1120

3. http://cpprocks.com/c11-compiler-support-shootout-visual-studio-gcc-clang-intel/



THANK
YOU!